The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Mohammad Yousuf	24BCE11172
Slot	**Date of Submission**
E11	31-03-2026

1. Introduction

Open source software is something that quietly powers a huge part of our daily digital life, even though most people dont really notice it. From the servers that host websites to the tools developers use and even the core of operating systems, a lot of it is built openly by communities instead of a single company. This idea of people from different parts of the world contributing together, sharing code, and improving it over time is what makes open source unique and honestly kind of amazing.

For this project, I worked on Linux, which itself is one of the biggest examples of open source in action. While using it, I realized that open source is not just about free software in terms of cost, but more about freedom to use, modify, study, and distribute the software. Open source software are also one of the most trusted software as it wont have any spyware that most software nowadays have. This concept might sound simple, but it has a huge impact on how technology grows and spreads.

The main goal of this project is to explore one real open source software project in detail. Instead of just explaining what the software does, I tried to look deeper into why it was created, who contributed to it, and what kind of ideas or philosophy it represents. Another important part is understanding its license, because that actually defines what we are allowed to do with the software. At first I thought licenses were just legal stuff, but they are actually very important in open source.

Overall, this project helped me understand that open source is not just a development method, but more like a mindset. It is about collaboration, transparency, and continuous improvement. Even though I am still learning, working on this made me appreciate how much of modern technology depends on contributions from people who choose to share their work with others.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: ~~Linux Kernel~~

3. Project Report Structure

A1. The problem this software, Linux was created to solve

When we talk about Linux today it feels like something that was always there.. That is not true. Linux started from a time when computers were costly hard to use and frustrating for users and students.

* Linux was not always available.

* Computing was Expensive.

Understanding this background is key. Otherwise Linux looks like another operating system.

Back in the 1980s and early 1990s most powerful operating systems were either owned by companies or had limited access. UNIX was a system. It was stable and efficient.. It was not free. The source code was. Required costly licenses. For a student or hobbyist this was a barrier.

There was MINIX. It was made for purposes. It taught operating system concepts. Many universities used it.. Minix had limitations. It was not fully open. It was not meant to be extended into a production-level system.

Linus Torvalds was a student at the University of Helsinki. He was interested in operating systems. He used MINIX.. He was not satisfied. He wanted something something he could control. He decided to build his system.

Linus did not start by saying "I will build the world’s popular operating system." He wanted a system that worked for him. That simplicity makes the story real. He started small. He focused on creating a kernel. He shared his work online.

The decision to share it openly was key. He posted about his project online. He asked for feedback and contributions. He allowed others to use it modify it and improve it.

Linux filled a piece. The GNU project had created tools.. It did not have a complete kernel.

Linux and GNU tools created an operating system.

Before Linux, operating systems were from companies. This meant control and high cost. Linux was different. Anyone could use it study it and modify it.

There was an issue. Proprietary software does not let users see how it works. Linux changed that. It gave users control.

Linux was adaptable. People used it on computers, servers and supercomputers.

Linux grew because people found it useful. Developers improved it. It was not controlled by a company.

There were challenges. Linux was not user-friendly. Installation was hard.. People used it for the freedom it provided.

Linux solved problems. It was freely available. Users could modify it. It was adaptable.

Linux is not just technical. It is a response to systems. It shows software can be built collaboratively and shared openly.

The impact of Linux is everywhere. From servers to Android phones modern technology depends on it. It started from a need and a decision to build something better.

The real problem Linux was created to solve was not just technical. It was about removing restrictions and giving control back, to users. That is what makes its origin story inspiring.

A2. The license — what it actually says

For this part of the project, I looked into the actual license used by Linux, which is the GNU General Public License (GPL), specifically version 2. At first I thought license is just some legal paragraph that nobody reads, but after going through it, I understood that it actually defines the entire idea of open source and how people are allowed to use the software.

One of the most important concepts in free software is the four freedoms. These are: the freedom to run the program for any purpose, the freedom to study how the program works, the freedom to modify it, and the freedom to distribute copies, including modified versions. Linux's license does support all four of these freedoms. Anyone can download the Linux kernel, use it however they want, check its source code, make changes, and even share their own modified version. This is what makes it truly "free" in the actual sense.

Now coming to the question of whether a company can take Linux, modify it, and sell it without sharing their changes. The answer is no, not completely. They can sell it, that is allowed, but if they distribute the modified version, they must also provide the source code and keep it under the same GPL license. This is because GPL is a "copyleft" license. It basically ensures that the software and any modifications of it remain open. So a company cannot just take Linux, improve it secretly, and sell it as a closed product. This rule keeps the ecosystem fair and transparent.

When comparing GPL with MIT license, there is a big difference. MIT license is much more permissive. It allows anyone to take the code, modify it, and even use it in proprietary software without sharing the changes. There are very few restrictions. On the other hand, GPL is stricter and forces anyone who distributes modified versions to also keep it open. So in simple words, MIT gives more freedom to developers, while GPL protects the freedom of the software itself.

If I had to choose a license for something I build, I think I would choose GPL, even though it is more restrictive. The reason is that it ensures my work stays open and cannot be turned into closed software by someone else. But at the same time, I understand why some people choose MIT, especially if they want their code to be used widely without any conditions.

There is also this interesting idea of "free as in free beer" vs "free as in freedom". At first it sounds confusing, but it actually makes sense. "Free as in free beer" means something you get without paying money. But "free as in freedom" means you have control over it, you can change it, share it, and understand how it works. Linux is a perfect example of "free as in freedom". Even if someone sells a Linux distribution, the source code is still available, and users still have all the rights given by the license.

Overall, studying the license made me realize that open source is not just about coding, it is also about rules and principles that protect user rights. Without licenses like GPL, open source software might not have grown the way it has today.

A3. The ethics of open source

Open source is not just about code being available, it also raises some important ethical questions about how software should be created and shared. While working with Linux, I started thinking whether all software should actually be open source or not. On one side, making everything open source sounds ideal because it promotes transparency, learning, and collaboration. Anyone can improve the software, fix issues, and there is less chance of hidden malicious things. But on the other side, not all companies or developers may want to reveal their work, especially if they invested a lot of time and money into it. So forcing everything to be open source might not be practical or fair in all situations.

Another interesting question is about companies making profit using open source software. For example, companies like Red Hat or Google use Linux in their systems and still earn money from it. At first it feels a bit unfair, like they are using community work for profit. But at the same time, these companies also contribute back in many ways, like improving the code, fixing bugs, and supporting the ecosystem. So I think it is not completely unethical, but it depends on whether they are giving back or just taking advantage.

The phrase “standing on the shoulders of giants” means building new things using the work that already exists. In software, this is very common. Linux itself was built with ideas from earlier systems, and now many modern technologies depend on Linux. Open source actually makes this easier because developers don’t have to start from zero every time. Some people might argue that it reduces originality, but I think it actually increases innovation because people can focus on improving instead of reinventing basic things again and again.

Overall, open source creates a balance between sharing and ownership. It is not perfect, but it definitely changes how we think about software and collaboration.

Part B — Linux Footprint (Unit 2)

For this section, I worked directly with the Linux kernel on my system and tried to understand how it actually exists and operates inside a Linux environment. Unlike normal software which you install and run like an application, the kernel is a core part of the operating system itself, so its presence is a bit different and not always visible in a simple way.

Talking about installation, the Linux kernel is usually not something you install manually in most cases. It comes pre-installed with the Linux distribution. But it can also be updated or reinstalled using package managers like apt in Ubuntu-based systems or RPM in Red Hat-based systems. For example, using apt, kernel updates are handled through system updates rather than installing it as a separate software. There is also an option to compile the kernel from source, which is more advanced and gives full control, but I did not go too deep into that as it can be risky if done wrong.

Now about directories, the kernel itself is stored in the /boot directory, usually as files like vmlinuz. The modules related to the kernel are stored in /lib/modules. Configuration files related to the system can be found in /etc, although the kernel itself does not rely on a single config file like normal applications. Logs related to system and kernel activities are stored in /var/log, especially in files like syslog or dmesg output.

When it comes to user and group, the kernel does not run as a normal user like other software. It operates at a very low level and has full control over the system. In a way, it runs with root-level privileges all the time. This is very important for security because if something goes wrong at the kernel level, it can affect the entire system. That is why kernel updates and modifications are handled very carefully.

For service management, the kernel itself is not started or stopped like a service. It is loaded during the boot process and remains active until the system is shut down. However, we can interact with it using commands like uname -r to check its version or dmesg to see kernel logs. Some kernel-related services or modules can be managed, but not the kernel directly like a normal service.

Regarding updates, the open-source community regularly provides updates and security patches for the Linux kernel. These updates are distributed through package managers. When I run system update commands, it includes kernel updates if available. This shows how open source community continuously works to fix vulnerabilities and improve the system. Unlike proprietary systems, these updates are transparent and often publicly documented.

Overall, working with the Linux kernel helped me understand that not all software behaves the same way on a Linux system. The kernel is deeply integrated and plays a critical role, and managing it requires more care compared to normal applications.

Part C — The FOSS Ecosystem (Units 3 & 4)

Linux is not one software it's a big ecosystem of many open source tools and projects that work together. When I was learning about this I realized that Linux alone isn't very helpful without the tools around it. It needs other parts like compilers, libraries and utilities to work properly For instance GNU tools like GCC, which's a compiler, Bash which is a shell and core utilities are really important Without these the Linux kernel can't be used effectively by users. So Linux and GNU are really connected even if they are projects. There are also libraries and system tools that rely on the kernel to work.

Things like glibc, which's the GNU C Library act as a bridge between applications and the kernel Programs don't directly interact with the kernel most of the time; they go through these libraries. This shows how everything is layered and connected not one standalone software. Linux has also inspired other projects and systems. One big example is Android, which uses the Linux kernel.

There are also Linux distributions like Ubuntu, Fedora and Arch Linux which are different versions built on top of the same kernel.

These distributions add their tools, package managers and user interfaces making Linux suitable for different types of users. So Linux has enabled a lot of innovation by acting as a base.

In terms of the LAMP stack, which includes Linux, Apache, MySQL and PHP Linux is the foundation. It acts as the operating system on which web servers like Apache run. Without Linux, the rest of the stack wouldn't work in the way. This makes Linux very important in the web world as many websites and servers depend on it. Today most of the internet infrastructure is powered by Linux-based systems.

There is also a community around Linux. Unlike software, where decisions are made by one company Linux development is managed by a global community. There are mailing lists where developers discuss changes report bugs and review code

There are also forums IRC channels and conferences where people share knowledge and collaborate. The Linux kernel is mainly coordinated by Linus Torvalds and other maintainers. Contributions come from thousands of developers worldwide. What I found interesting is that there is no owner" of Linux but its still highly organized

Different parts of the kernel are maintained by experts and changes go through a review process before being accepted. This ensures quality while keeping it open.

Overall the ecosystem around Linux shows that open source is not about one project but about how multiple projects connect and grow together. It's, like a network where each part supports the other. That is what makes it so powerful.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source: Linux	Proprietary Alternative: Windows
Cost	Free	Paid
Security (who can audit the code?)	Open audit	Closed audit
Support and reliability	Community	Official
Freedom to modify	Free to do any modification	Way too restricted

Community vs corporate control	Community driven	Corporate control, focuses on money insted of ease for user
Your overall verdict	Linux best for people who want flexibility	Good for people who dont have IT background.

After going through all these aspects, I think Linux is definitely suitable for real-world deployment, especially in areas like servers, development environments, and cybersecurity. It provides strong stability, better control, and transparency which is very important in professional systems. But at the same time, for normal everyday users who just want simplicity and ease, it might not always be the first choice. So I would say Linux is highly recommended, but depending on the use case and user comfort level.

As for contributing to it, I would like to, even though right now my skills are still developing. The idea that anyone can be a part of improving such a big system is honestly very motivating. Even small contributions like fixing bugs or improving documentation can make a difference. In future, once I get more confident in my coding and understanding, I would definitely try to contribute properly instead of just using it.

4. Shell Script Tasks

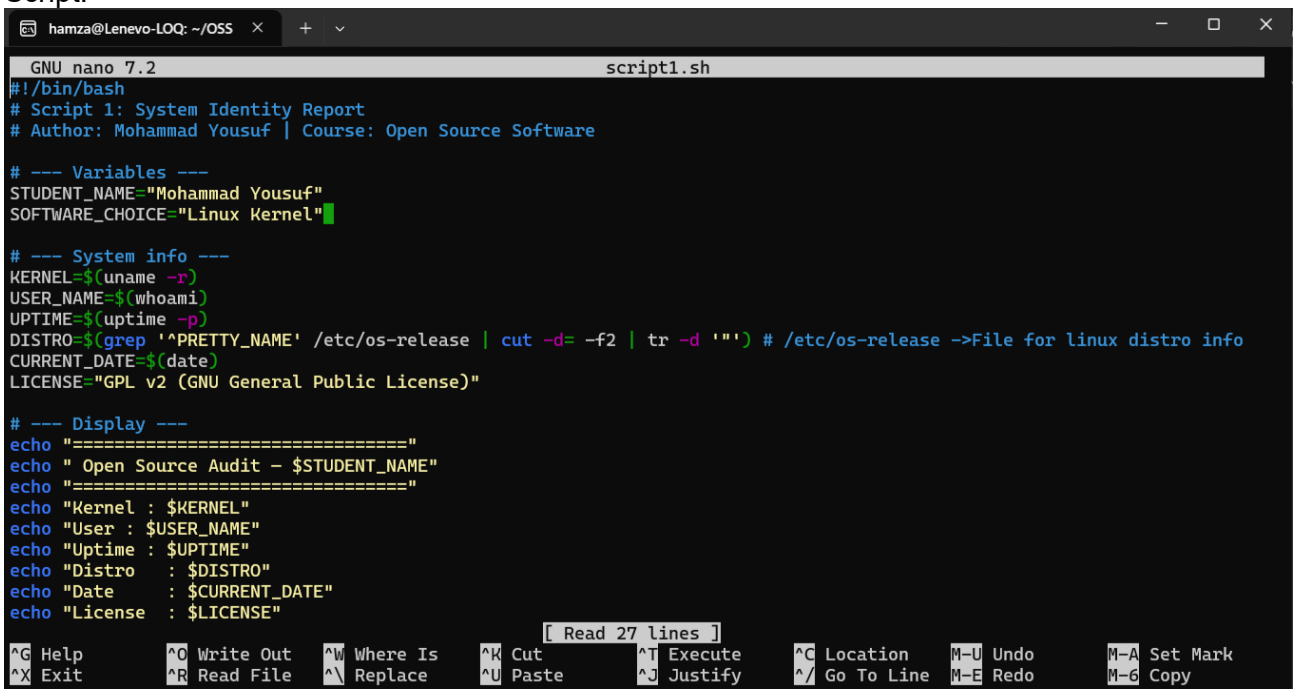
Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

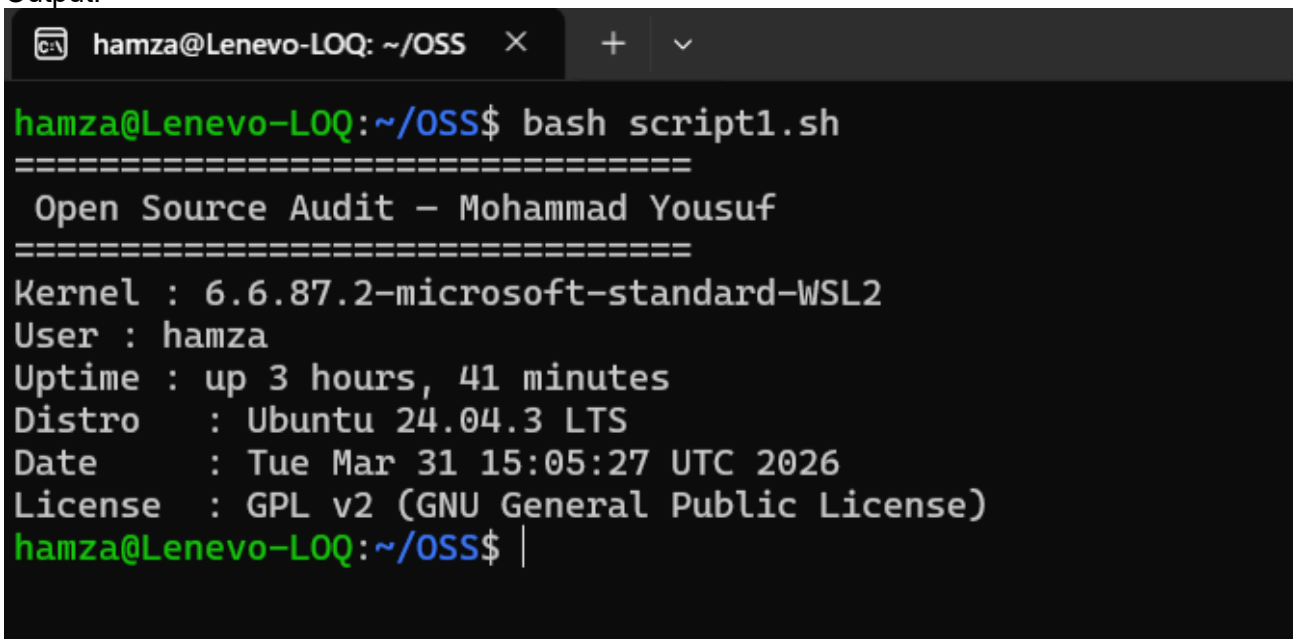
- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Script:



Output:

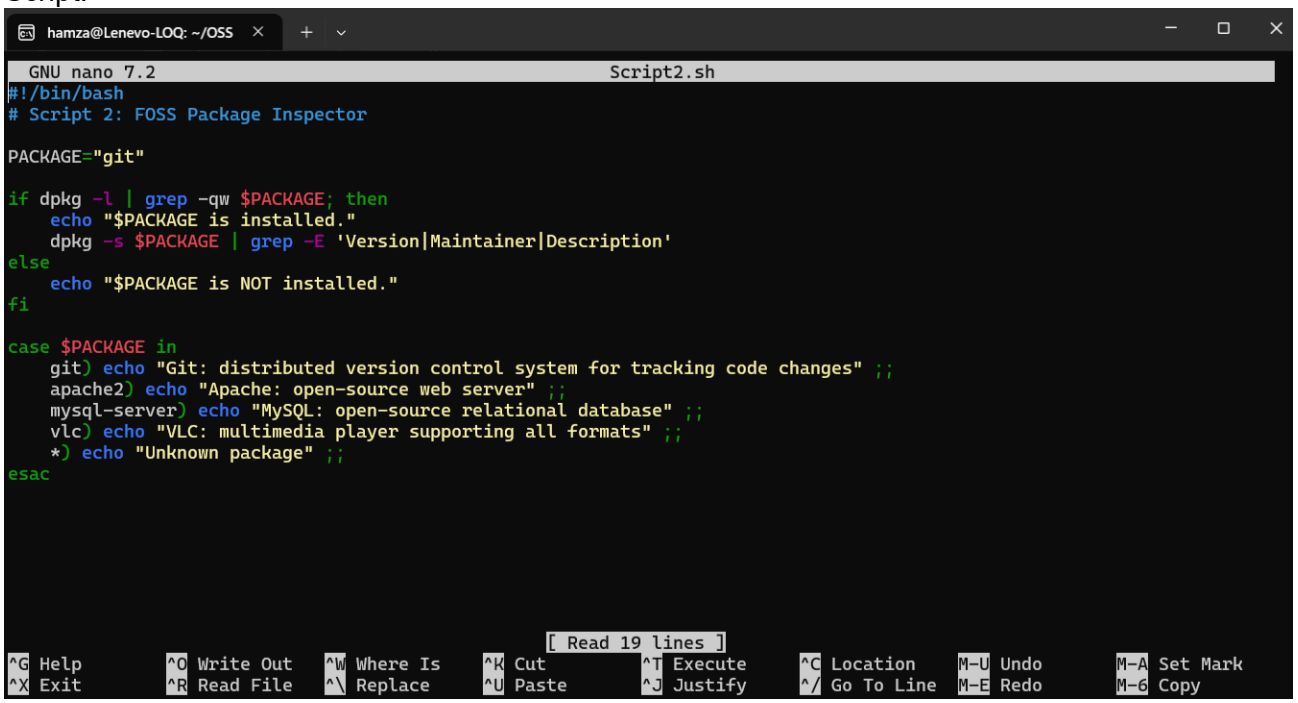


Script 2 — FOSS Package Inspector (Unit 2)

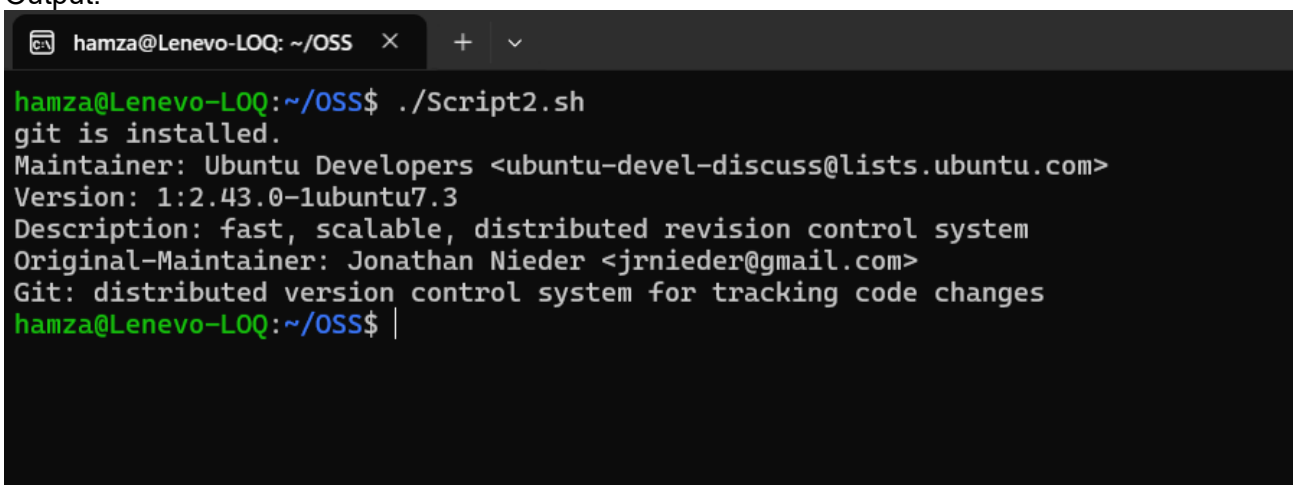
Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Script:



Output:



Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Script:

```
hamza@Lenevo-LOQ: ~/OSS
  GNU nano 7.2                                  script3.sh
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# Check Git config directory
if [ -d "/etc/git" ]; then
    echo "Git config found:"
    ls -ld /etc/git
else
    echo "Git config directory not found"
fi

                        [ Read 25 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute   ^C Location  M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

Output:

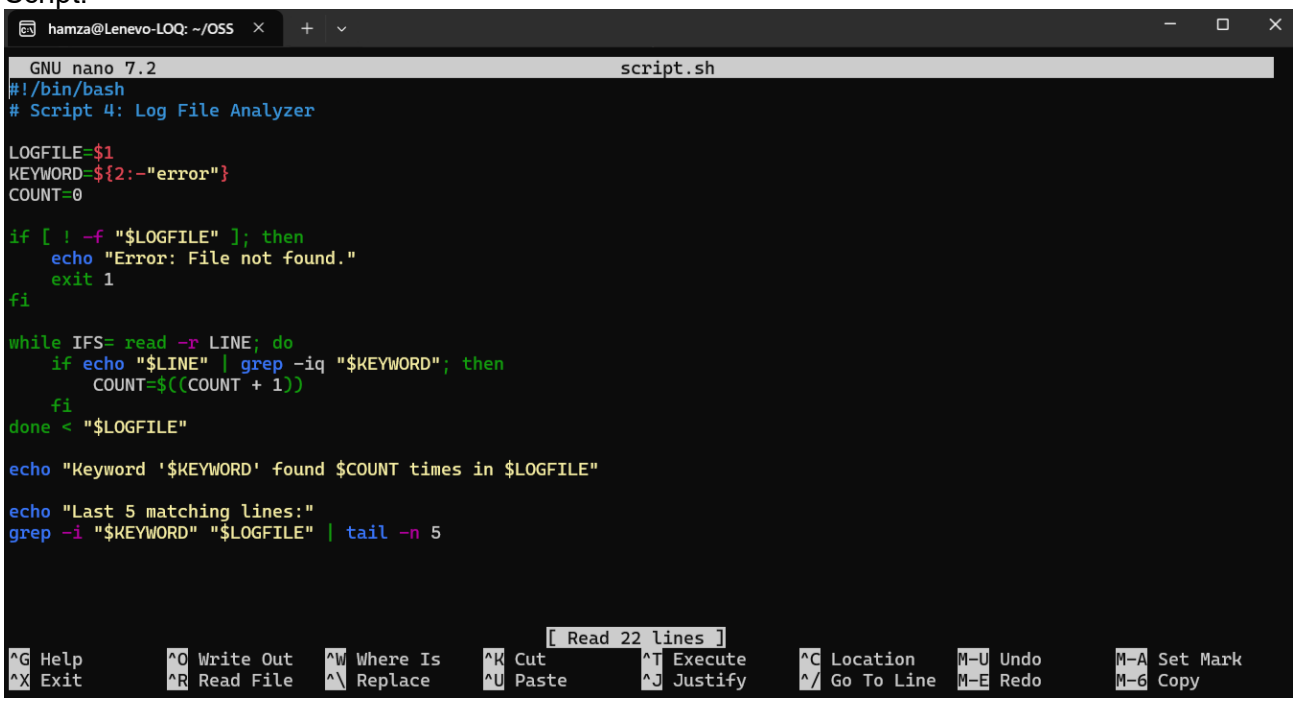
```
hamza@Lenevo-LOQ: ~/OSS
hamza@Lenevo-LOQ:~/OSS$ bash script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.5M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 429M
/home => Permissions: drwxr-xr-x root root | Size: 741M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 134M
/tmp => Permissions: drwxrwxrwt root root | Size: 36K
Git config directory not found
hamza@Lenevo-LOQ:~/OSS$
```

Script 4 — Log File Analyzer (Units 2 & 5)

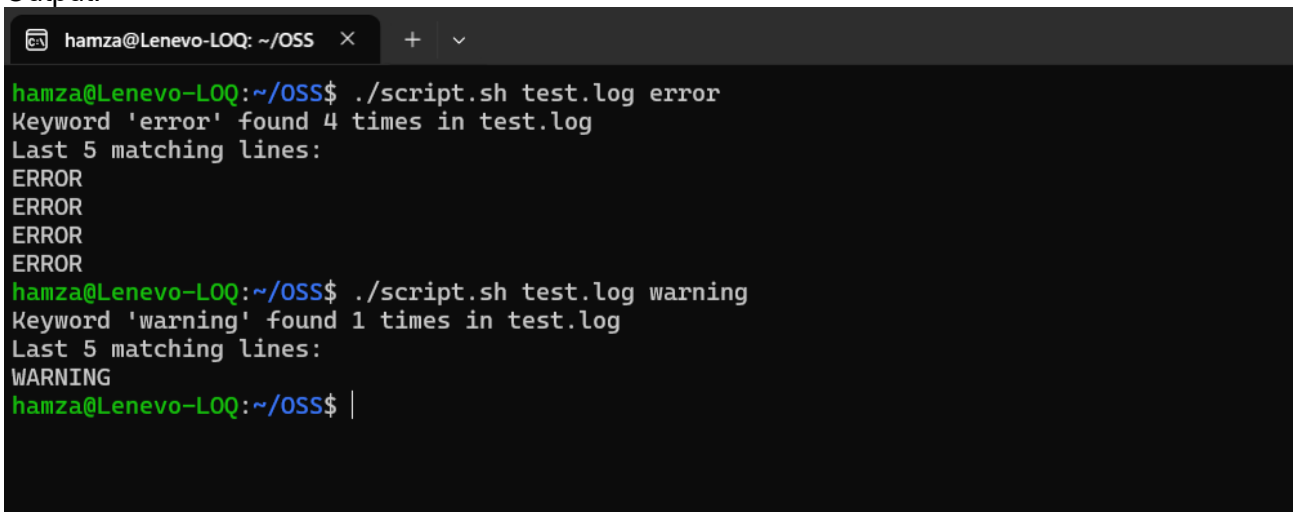
Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Script:



Output:



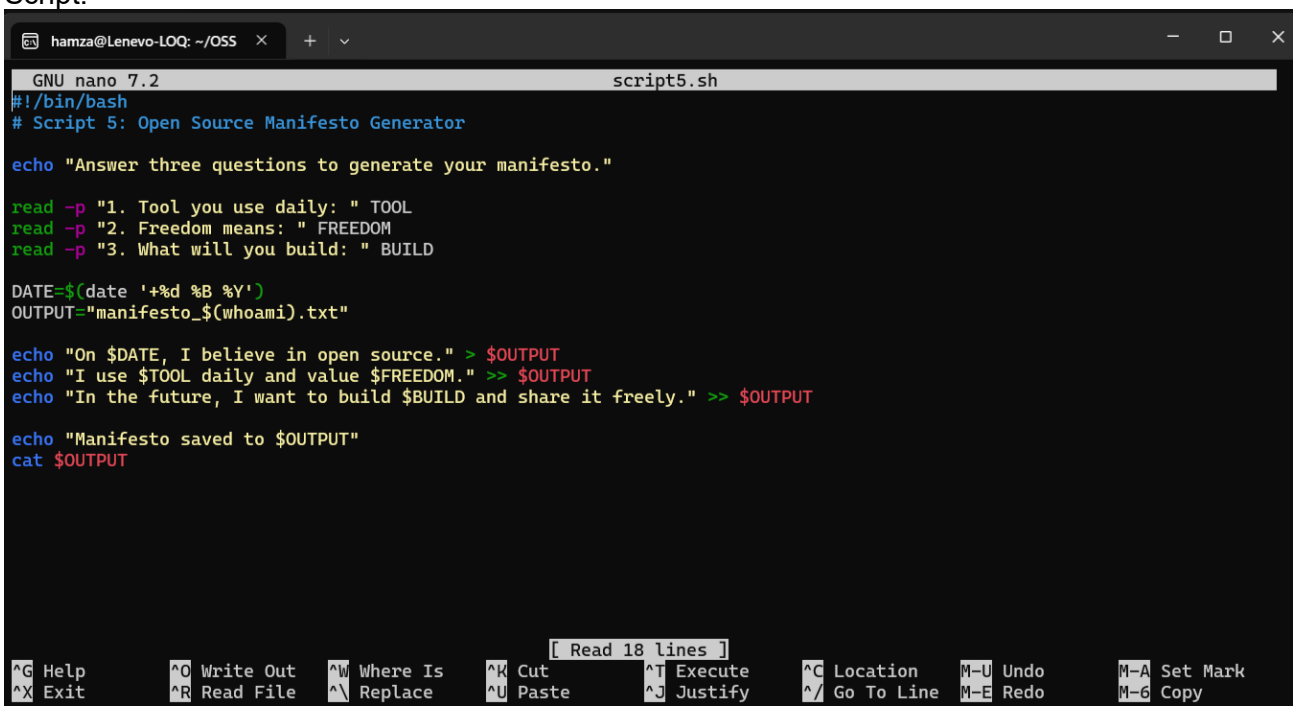
Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source

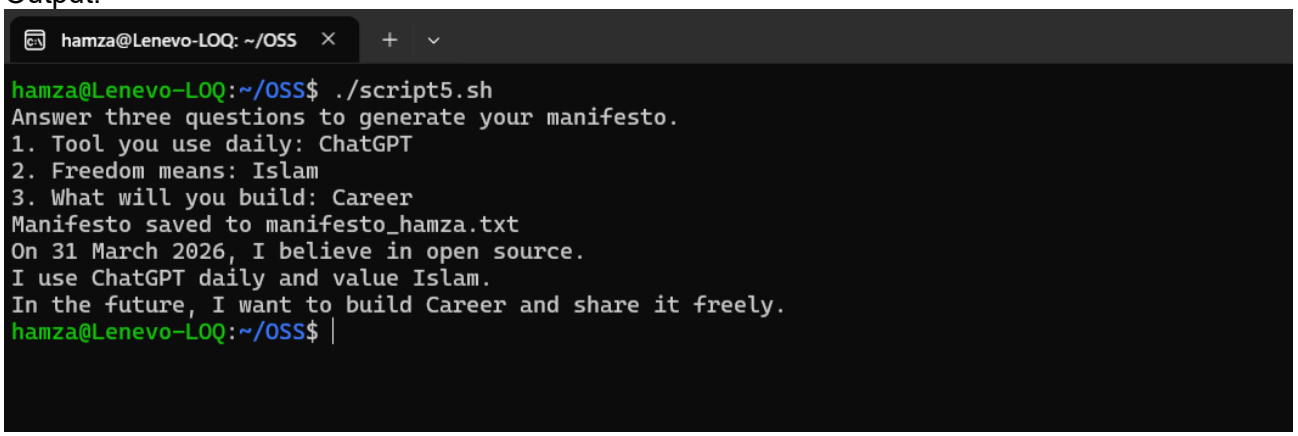
philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Script:



Output:



5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	**12**	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	**12**	6	*Part A2*
3	Ethical reflection — quality of argument and original thinking	**10**	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	**10**	5	*Part B*
5	Ecosystem and LAMP connection	**8**	4	*Part C*
6	Open source vs proprietary comparison	**8**	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	**4**	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	**2**	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	**2**	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.